

Week 2 - Day 6b Supplementary: Loops and Modules Challenge!

Run the following cell before doing the exercises.

```
# Run this function
def check(fn, *input, expected):
    result = fn(*input)
    if expected != result:
        print(
            f"Function should return the value {expected}, it is"
            f"returning the value {result}."
        )
    else:
        print(f"Congratulations, the test case passed!")
```

Exercise 1: Characters and Distances []

1.1 You are given a list of strings `words` and a character `x`. Return an array of numbers representing the number of times each word contains the character `x`.

Input: `words = ["addis", "coder", "summer"], x = "d"` Output: `[2, 1, 0]`
Explanation: "d" occurs in "addis" twice, in "coder" once, and in "summer" 0 times.

```
# TODO: Write your code here
```

1.2 Given a string `s` and a character `c` that occurs in `s`, implement a function that returns an array of integers `answer` where `answer[i]` is the closest distance from index `i` to the occurrence of character `c` in `s`.

Input: `s = "aeebcdefgz", c = "e"`

Output: `[1,0,0,1,2,1,0,1,2,3]`

```
# TODO: Write your code here
```

Exercise 2: Patterns []

2.1 Write a function that takes a number `n` and prints the following pattern up to `n`: Input: `n = 4` Output:

```
1
2 2
3 3 3
4 4 4 4
```

```
# TODO: Write your code here
```

2.2 Write a function that takes a number `n` and prints the following pattern up to `n`: Input: `n = 4` Output:

```
*
***
*****
*****
```

```
# TODO: Write your code here
```

2.3 Write a function that prints the following pattern using a loop:

```
A
B C
D E F
G H I J
K L M N O
```

```
# TODO: Write your code here
```

2.4 Given an input number `n`, write a function that prints the following pattern `n` times.

```
A
B B
C C C
D D D D
```

```
# TODO: Write your code here
```

2.5 Write a function that takes a number `n` and prints the following pattern up to `n`: Input: `n = 4` Output:

```
****
***
**
*
```

```
# TODO: Write your code here
```

2.6 Write a function that prints the following pattern using loops: Input : `n = 5` Output:

```
*
* *
* * *
* * * *
* * * * *
```

```
* * * *  
* * *  
* *  
*
```

TODO: Write your code here

Exercise 3: Modules []

3.1 Calculate multiplication of two random float numbers between 1 and 10

TODO: Write your code here

3.2 Generate and print 3 random integers between 100 and 999 which are divisible by 7.

TODO: Write your code here

Exercise 4: Unique Occurrences []

Write a function `print_stat` that takes a list of numbers as input and prints each unique number in the list, along with the number of times it appears in the list. If a number does not appear in the list, it should be printed without any count.

Example:

- Input: [15, 4, 7, 2, 2, 9, 1, 2, 5, 4]
- Output:

```
1 : 1  
2 : 3  
3  
4 : 2  
5 : 1  
6  
7 : 1  
8  
9 : 1  
10  
11  
12  
13  
14  
15 : 1
```

TODO: Write your code here

Exercise 5: Dice Challenge []

5.1 Abdishakur wants to know how many times he needs to roll a six-sided die until it lands on six. Help Abdishakur count the number of rolls it takes. Write a function `count_dice_rolls()` that rolls the die (using `random.randrange(1, 7)`) until a six is rolled, and returns the number of rolls needed.

```
# TODO: write your code here
```

Abdishakur noticed that the result changes each time you call the function. He now wants to know the average number of rolls needed and visualize the distribution of outcomes.

5.2 Write a function `compute_average` that calls the function `count_dice_rolls()` 10,000 times to compute the average number of dice rolls.

Example: if we roll 10 times, we might get `[16, 2, 2, 9, 5, 2, 5, 1, 7, 5]`. The average number of rolls is `5.4`.

```
# TODO: Write your code here
```

5.3 Write a function `visualize_dist` that calls the function `count_dice_rolls()` 10,000 times and uses your function `print_stat` to visualize the distribution of outcomes.

Example: if we roll 10 times, we might get `[10, 2, 2, 9, 5, 2, 5, 1, 7, 5]`. Your output should be:

```
1:1
2:3
3
4
5:3
6
7:1
8
9:1
10:1
```

```
# TODO: Write your code here
```

Exercise 6: Random Draws []

6.1 We can generate a random number between 0 and 1 using the function `random.random()`. Write a function `count_random_draws(p)` that takes `p` as an parameter and that generates random numbers until one of them is larger than `p`. Return the number of random numbers that were generated until one of them is larger than `p`.

```
# TODO: Write your code here
```

6.2 As you did in 5.2, write a function that calls `count_random_draws(0.8)` 10,000 times to compute the average number of random draws.

```
# TODO: Write your code here
```

6.3 As you did in 5.3, write a function that calls `count_random_draws(0.8)` 10,000 times and uses your function `print_stat` to visualize the distribution of outcomes.

```
# TODO: Write your code here
```

Exercise 7: Randomly Adding to a Sum []

7.1 Write a function `count_until_sum(n)` that takes a parameter `n` and generates random numbers between 0 and 1 until their *sum* exceeds `n`. Return how many numbers were generated.

```
# TODO: Write your code here
```

7.2 As you did in 5.2, write a function that calls `count_until_sum(10)` 10,000 times to compute the average number of iteration it takes until sum exceeds `n`.

```
# TODO: Write your code here
```

7.3 As you did in 5.3, write a function that calls `count_until_sum(10)` 10,000 times and uses your function `print_stat` to visualize the distribution of outcomes.

```
# TODO: Write your code here
```

Exercise 8: Lucky Lottery Draw []

Random Lottery Pick. Generate 100 random lottery tickets and pick two lucky tickets from it as a winner. Note

- The lottery number must be 10 digits long.
- All 100 ticket numbers must be unique.

```
# TODO: Write your code here
```

Exercise 9: Permutation []

Write a function that generates all possible unique permutations of the characters in a given string. The string will only contain alphanumeric characters (no special characters or spaces), and letters should be treated case-insensitively (i.e., 'A' and 'a' are considered the same and should not appear together in any permutation). The function should return a list of all unique permutations.

9.1 Start by writing a function that takes a string `s` and returns an uppercase string that contains unique letters and doesn't contain spaces and special characters.

Input: `s = "Abebe_&1Ababa"` Output: `ABE`

```
# TODO: Write your code here
```

9.2 Write a function that takes the output of the above function as an input and prints all unique permutations.

Input: `"ABE"` Output:

```
"ABE"  
"AEB"  
"BAE"  
"BEA"  
"EAB"  
"EBA"
```

```
# TODO: Write your code here
```

Congratulations 🎉